САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №1
по курсу «Алгоритмы и структуры данных»
Тема: Графы
Вариант 16

Выполнила:
Бархатова Н.А.
К3139

Проверила:
Артамонова В.Е.

Санкт-Петербург
2023 г.

Содержание отчета

Содержание отчета 2
Задачи по варианту 3
Задача №1. Лабиринт [1 балл] 3
Задача №9. Аномалии курсов валют [2 балла] 5
Задача №16. Рекурсия [3 балла] 7
Дополнительные задачи 8
Задача №2. Компоненты [1 балл] 8
Задача №3. Циклы [1 балл] 10
Задача №4. Порядок курсов [1 балл] 13
Вывод 15

Задачи по варианту

Задача №1. Лабиринт [1 балл]

Текст задачи.

Лабиринт представляет собой прямоугольную сетку ячеек со стенками между некоторыми соседними ячейками.

Вы хотите проверить, существует ли путь от данной ячейки к данному выходу из лабиринта, где выходом также

является ячейка, лежащая на границе лабиринта (в примере, показанном на рисунке, есть два выхода: один на левой

границе и один на правой границе). Для этого вы представляете лабиринт в виде неориентированного графа: вершины

графа являются ячейками лабиринта, две вершины соединены неориентированным ребром, если они смежные и между

ними нет стены. Тогда, чтобы проверить, существует ли путь между двумя заданными ячейками лабиринта, достаточно

проверить, что существует путь между соответствующими двумя вершинами в графе.

Листинг кода.

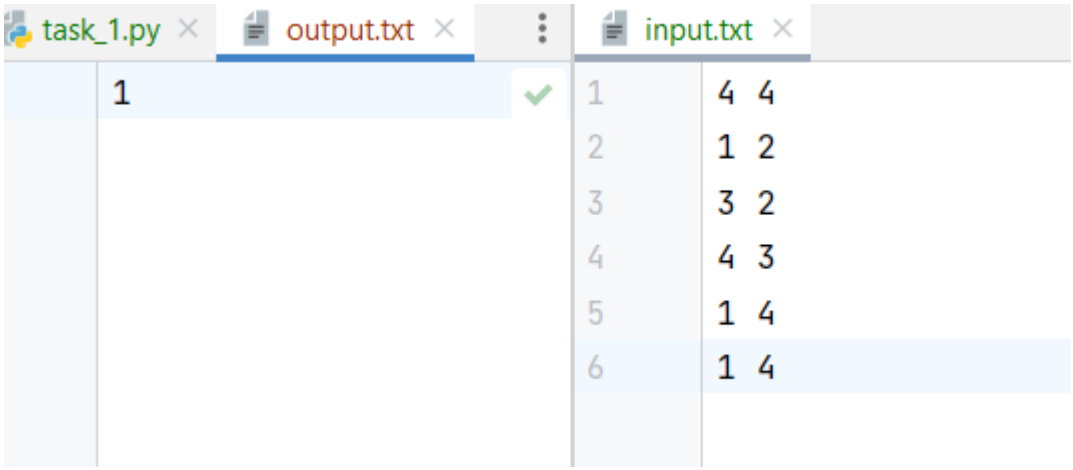
```
import time
import tracemalloc

start_time = time.perf_counter()
tracemalloc.start()
input_file = open('input.txt')
vertices, edges = map(int, input_file.readline().split())
graph = {}
for i in range(edges):
    from_vertex, to_vertex = map(int, input_file.readline().split())
    if from_vertex not in graph.keys():
        graph[from_vertex] = []
    if to_vertex not in graph.keys():
        graph[to_vertex] = []
    graph[from_vertex].append(to_vertex)
    graph[to_vertex].append(from_vertex)
u, v = map(int, input_file.readline().split())
visited = [False] *(vertices+1)
order = []
def explore(v):
    order.append(v)
    visited[v] = True
    for w in graph[v]:
        if not visited[w]:
            explore(w)
def DFS(graph, start):
    for v in list(graph.keys())[start-1:]:
```

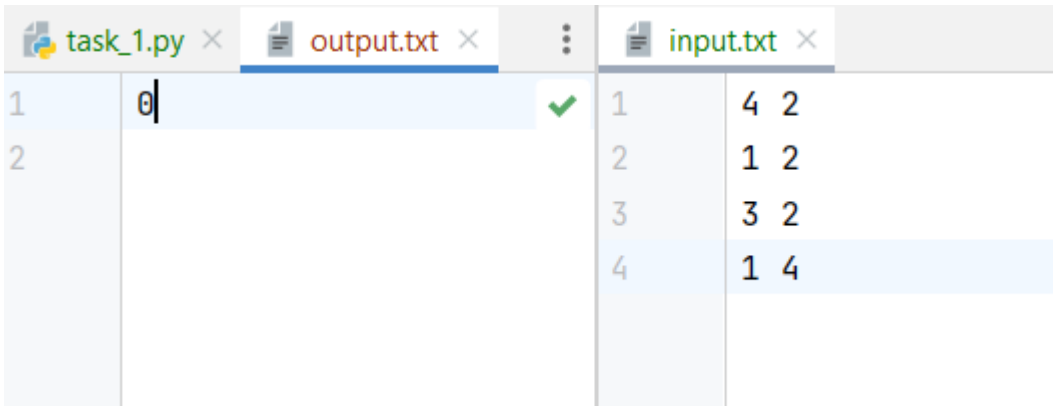
```
        if not visited[v]:
            explore(v)
    return order
with open('output.txt', 'w') as output_file:
    print(int(v in DFS(graph, u)), file=output_file)
print("Время: ", time.perf_counter() - start_time)
print("Память: ", float(tracemalloc.get_tracemalloc_memory()) / (2 ** 20))
tracemalloc.stop()
```

Текстовое объяснение решения.

Считываю input.txt и задаю граф как список смежностей. Создаю функцию обхода графа. Если во время обхода графа с первой вершины попадается вторая, то значит, они соединены между собой.

Результат работы кода на примерах из текста задачи:(скрины input output файлов)





	Время выполнения	Затраты памяти
Пример из задачи	0.0010	0.0037
Пример из задачи	0.0008	0.0037

Вывод по задаче: легко и просто

Задача №9. Аномалии курсов валют [2 балла]

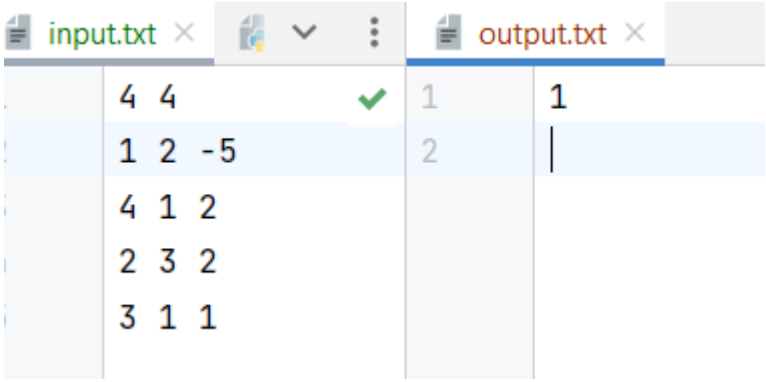
Текст задачи.
Для заданного ориентированного графа с возможными отрицательными весами ребер, у которого n вершин и m ребер, проверьте, содержит ли он цикл с отрицательным суммарным весом.

Листинг кода.

```
import time
import tracemalloc


def bellman_ford(n, S, edges):
    dist = [float('inf')] * (n + 1)
    dist[S] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
                no_inf_dist = [i for i in dist if isinstance(i, int)]
                if sum(no_inf_dist) < 0:
                    return 1
    return 0


start_time = time.perf_counter()
tracemalloc.start()
input_file = open('input.txt')
vertices, edges = map(int, input_file.readline().split())
values = [tuple(map(int, input_file.readline().split())) for _ in
range(edges)]
with open('output.txt', 'w') as output_file:
    print(bellman_ford(vertices, 1, values), file=output_file)
print("Время: ", time.perf_counter() - start_time)
print("Память: ", float(tracemalloc.get_tracemalloc_memory()) / (2 ** 20))
tracemalloc.stop()
```

Текстовое объяснение решения:
Реализуем алгоритм Беллмана-Форда

Результат работы кода на примерах из текста задачи:(скрины input output файлов)



	Время выполнения	Затраты памяти
Пример из задачи	0.0008	0.0034

Вывод по задаче: выглядит сложнее, чем есть на самом деле

Задача №16. Рекурсия [3 балла]

Текст задачи.
Рассмотрим программу, состоящую из n процедур P1, P2, . . . , Pn. Пусть для каждой процедуры известны процедуры, которые она может вызывать. Процедура P называется потенциально рекурсивной, если существует такая последовательность процедур Q0, Q1, . . . , Qk, что Q0 = Qk = P и для i = 1...k процедура Qi−1 может вызвать процедуру Qi. В этом случае задача будет заключаться в определении для каждой из заданных процедур, является ли она потенциально рекурсивной. Требуется написать программу, которая позволит решить названную задачу.

Листинг кода.

```
import time
import tracemalloc

start_time = time.perf_counter()
tracemalloc.start()
input_file = open('input.txt')
n = int(input_file.readline().strip())
graph = {}
for _ in range(n):
    p_id = input_file.readline().strip()
    p_calls = int(input_file.readline().strip())
    graph[p_id] = []
    if p_calls != 0:
        for _ in range(p_calls):
            called_prog = input_file.readline().strip()
            graph[p_id].append(called_prog)
    input_file.readline()


def dfs(v, visited, start_v):
    if v in visited:
        return v == start_v
    visited.append(v)
    for neighbor in graph[v]:
        if dfs(neighbor, visited, start_v):
            return True
    return False


result = ''

for p_id in list(graph.keys()):
    visited = []
    if dfs(p_id, visited, p_id) and graph[p_id] is not None:
        result += 'YES\n'
    else:
        result += 'NO\n'

with open('output.txt', 'w') as output_file:
    print(result, file=output_file)
print("Время: ", time.perf_counter() - start_time)
print("Память: ", float(tracemalloc.get_tracemalloc_memory()) / (2 ** 20))
tracemalloc.stop()
```

Текстовое объяснение решения.
Считываем данные в ориентированный граф. Определяем для каждой вершины её достижимость из самой же себя.

Результат работы кода на примерах из текста задачи:(скрины input output файлов)

input.txt

```
3
p1
2
p1
p2
*****
p2
1
p1
*****
p3
1
p1
*****
```

task_16.py acmp_check.py output.txt

```
YES
YES
NO
```

Проверка задачи на (openedu, acmp и тд при наличии в задаче). (скрин)

№1 - №20 Вперед »

ID	Дата	Автор	Задача	Язык	Результат	Тест	Время	Память
19533913	13.06.2023 0:20:26	Бархатова Наталья Александровна	0345	Python	Accepted		0,062	1694 Кб
19533912	13.06.2023 0:20:21	Арслан Гапт	1418	Python	Wrong answer	13	0,046	466 Кб
19533911	13.06.2023 0:18:36	Беляков Семён Вадимовіч	0295	Python	Wrong answer	1		494 Кб
19533910	13.06.2023 0:16:01	Кооп Максим	0867	Python	Wrong answer	3	0,031	486 Кб

	Время выполнения	Затраты памяти
Пример из задачи	0.0013	0.0042

Вывод по задаче: при решении используется алгоритм из первых задач этой лабораторной работы.

Дополнительные задачи

Задача №2. Компоненты [1 балл]

Текст задачи.

Теперь вы решаете сделать так, чтобы в лабиринте не было мертвых зон, то есть чтобы из каждой клетки был
доступен хотя бы один выход. Для этого вы находите связные компоненты соответствующего неориентированного
графа и следите за тем, чтобы каждый компонент содержал выходную ячейку.
Дан неориентированный граф с n вершинами и m ребрами. Нужно посчитать количество компонент связности в
нем.

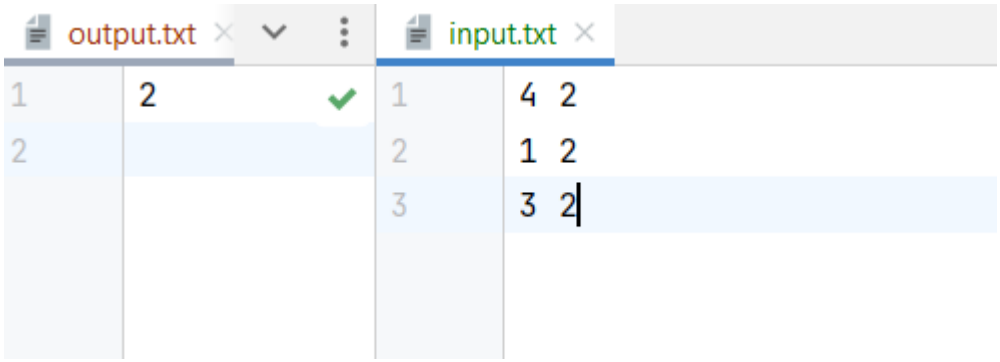
Листинг кода.

```
import time
import tracemalloc

start_time = time.perf_counter()
tracemalloc.start()
input_file = open('input.txt')
vertices, edges = map(int, input_file.readline().split())
graph = {}
for i in range(edges):
    from_vertex, to_vertex = map(int, input_file.readline().split())
    if from_vertex not in graph.keys():
        graph[from_vertex] = []
    if to_vertex not in graph.keys():
        graph[to_vertex] = []
    graph[from_vertex].append(to_vertex)
    graph[to_vertex].append(from_vertex)
while len(graph.keys()) != vertices:
    index = 1
    for i in range(1, len(graph.keys()) + 2):
        if i not in graph.keys():
            index = i
            break
    graph[index] = []
visited = [False] * (vertices + 1)
order = []
CC_nums = [0] * (vertices + 1)
cc_number = 1


def explore(v):
    CC_nums[v] = cc_number
    order.append(v)
    visited[v] = True
    for w in graph[v]:
        if not visited[w]:
            explore(w)


def DFS(graph, start):
    global cc_number
    for v in list(graph.keys())[start - 1:]:
        if not visited[v]:
            explore(v)
```

```
            cc_number += 1

DFS(graph, 1)
with open('output.txt', 'w') as output_file:
    print(max(CC_nums), file=output_file)
print("Время: ", time.perf_counter() - start_time)
print("Память: ", float(tracemalloc.get_tracemalloc_memory()) / (2 ** 20))
tracemalloc.stop()
```

Текстовое объяснение решения.
Запускаю обход графа от вершины с номером 1. Если обход прервался, но при этом не посещенные вершины всё ещё есть, значит, при прерывании был завершен обход компоненты.

Результат работы кода на примерах из текста задачи:(скрины input output файлов)



	Время выполнения	Затраты памяти
Пример из задачи	0.0023	0.0040

Вывод по задаче: весело

Задача №3. Циклы [1 балл]

Текст задачи.
Проверьте, содержит ли данный граф циклы.

Листинг кода.

```
import time
import tracemalloc

start_time = time.perf_counter()
tracemalloc.start()
input_file = open('input.txt')
vertices, edges = map(int, input_file.readline().split())
```

```
graph = {}
for i in range(edges):
    from_vertex, to_vertex = map(int, input_file.readline().split())
    if from_vertex not in graph.keys():
        graph[from_vertex] = []
    if to_vertex not in graph.keys():
        graph[to_vertex] = []
    graph[from_vertex].append(to_vertex)
queue = []
visited = [False] * (vertices + 1)
colors = ['white' for _ in range(vertices + 1)]


def dfs(start, visited, graph):
    visited[start] = True
    colors[start] = 'grey'
    for v in graph[start]:
        if colors[v] != 'grey':
            dfs(v, visited, graph)
        else:
            raise Exception
    colors[start] = 'black'


with open('output.txt', 'w') as output_file:
    try:
        dfs(1, visited, graph)
        print(0, file=output_file)
    except:
        print(1, file=output_file)

print("Время: ", time.perf_counter() - start_time)
print("Память: ", float(tracemalloc.get_tracemalloc_memory()) / (2 ** 20))
tracemalloc.stop()
```

Текстовое объяснение решения.

В случае ориентированного графа произведём серию обходов. То есть из каждой вершины, в которую мы ещё ни разу не приходили, запустим поиск в глубину, который при входе в вершину будет красить её в серый цвет, а при выходе из нее — в чёрный. И, если алгоритм пытается пойти в серую вершину, то это означает, что цикл найден.

Результат работы кода на примерах из текста задачи:(скрины input output файлов)

input.txt | task_3.py | output.txt

```
5 7
1 2
2 3
1 3
3 4
1 4
2 5
3 5
```

```
0
```

input.txt | task_3.py | output.txt

```
4 4
1 2
4 1
2 3
3 1
```

```
1
```

	Время выполнения	Затраты памяти
Пример из задачи	0.0013	0.0038

Вывод по задаче: весело

Задача №4. Порядок курсов [1 балл]

Текст задачи.

Дан ориентированный ациклический граф (DAG) с n вершинами и m ребрами. Выполните топологическую сортировку.

Листинг кода.

```
import time
import tracemalloc

start_time = time.perf_counter()
tracemalloc.start()
input_file = open('input.txt')
vertices, edges = map(int, input_file.readline().split())
graph = {}
for i in range(edges):
    from_vertex, to_vertex = map(int, input_file.readline().split())
    if from_vertex not in graph.keys():
        graph[from_vertex] = []
    if to_vertex not in graph.keys():
        graph[to_vertex] = []
    graph[from_vertex].append(to_vertex)
graph = dict(sorted(graph.items()))
order = []
pre = [0 for _ in range(vertices + 1)]
post = [0 for _ in range(vertices + 1)]
visited = [False for _ in range(vertices + 1)]
clock = 1


def explore(v):
    order.append(v)
    visited[v] = True
    global clock
    pre[v] = clock
    clock += 1
    for w in graph[v]:
        if not visited[w]:
            explore(w)
    post[v] = clock
    clock += 1


def DFS(graph):
    for v in graph.keys():
        if not visited[v]:
            explore(v)


DFS(graph)
node_and_post = list(zip(order, [post[i] for i in order]))
topological_order = [i[0] for i in sorted(node_and_post, reverse=True,
key=lambda x: x[1])]
with open('output.txt', 'w') as output_file:
    print(*topological_order, file=output_file)
print("Время: ", time.perf_counter() - start_time)
print("Память: ", float(tracemalloc.get_tracemalloc_memory()) / (2 ** 20))
tracemalloc.stop()
```

Текстовое объяснение решения.

Вводим метки времени. Ответом будет являться порядок вершин, отсортированный по массиву post в порядке убывания.

Результат работы кода на примерах из текста задачи:(скрины input output файлов)

input.txt

```
4 3
1 2
4 1
3 1
```

task_4.py output.txt

```
4 3 1 2
```

input.txt

```
4 1
3 1
```

output.txt task_4.py

```
4 2 3 1
```

input.txt

```
5 7
2 1
3 2
3 1
4 3
4 1
5 2
5 3
```

task_4.py output.txt

```
5 4 3 2 1
```

	Время выполнения	Затраты памяти
Пример из задачи	0.0028	0.0049
Пример из задачи	0.0012	0.0062

Вывод по задаче: класс

Вывод

В результате выполнения данной лабораторной работы я ознакомилась с основными принципами работы с графами, алгоритмами поиска путей в графах. Таким образом, выполнение данной лабораторной работы позволило получить полное представление об основных алгоритмах и структурах данных, используемых в работе с графами, что будет полезно при дальнейшем изучении и применении этой темы в практических задачах.